Shipyard Pre-Search

2026-03-23 — Agent Architecture & Design Decisions

This document answers the Shipyard Pre-Search checklist directly. It is grounded in the Shipyard PRD and the local Ship monorepo. The result is a TypeScript coding agent built on LangGraph.js with anchor-based surgical editing, a single coordinator writer, read-only subagents, and LangSmith tracing — designed for the Ship monorepo.

R FINAL RECOMMENDATION

I will build a TypeScript coding agent that runs in a persistent local Node process, uses Claude Sonnet 4.5 as the default model, uses LangGraph.js for orchestration, and uses LangSmith for traces. The agent uses anchor-based surgical editing rather than full-file rewrites. A single coordinator is the only writer, while subagents handle read-only research and verification in parallel.

• ARCHITECTURE

Runtime: Persistent local Node.js process	**Model:** Claude Sonnet 4.5 (default)
Orchestration: LangGraph.js	**Tracing:** LangSmith
Editing: Anchor-based surgical replacement	**Writing:** Single coordinator only

Three key design decisions shape the multi-agent coordination model described in section 5: pass small typed artifacts between stages instead of free-form agent chatter, isolate planning from editing from verification, and verify inside the loop instead of waiting until the end.

1 AGENTS STUDIED

I studied three agents in depth: OpenCode, Open SWE, and Claude Code. For each, I examined how it handles file editing, how it manages context across turns, and how it handles failed tool calls and unexpected output. Each agent excels at a different dimension, and my architecture borrows from all three.

OpenCode

BEST FOR SURGICAL EDITING

FILES STUDIED

edit.go · patch.go · agent.go · agent-tool.go · README.md

HOW IT EDITS

Uses exact string replacement through an edit tool and structured multi-file patch application through a patch tool. Rejects ambiguous matches and patch application when context is too fuzzy. Checks whether a file changed after it was last read.

CONTEXT MANAGEMENT

Auto-compacts when the conversation approaches the context window limit. Creates a summarized continuation session instead of failing on long conversations.

ERROR HANDLING

Returns explicit tool errors for ambiguous edits, stale reads, bad paths, and patch parse failures.

WHAT I'D TAKE

› Exact-match editing · › Atomic patch application · › Stale-read protection · › Automatic context compaction

WHAT I'D CHANGE

› Keep subagents read-only · › Add stronger post-edit verification · › Simpler primary interface than full patch text

Open SWE

BEST FOR ORCHESTRATION

FILES STUDIED

server.py · check_message_queue.py · tool_error_handler.py · open_pr.py · agents_md.py

HOW IT EDITS

Delegates file editing to Deep Agents filesystem tools. Uses string-based edit_file and write_file primitives.

CONTEXT MANAGEMENT

Injects repo rules from AGENTS.md. Injects issue or thread context from the invocation surface. Keeps a persistent per-thread sandbox. Queues follow-up messages and injects them before the

next model call. Uses summarization and prompt caching through Deep Agents middleware.

ERROR HANDLING

Converts tool exceptions into structured tool errors instead of crashing the run. Uses deterministic middleware backstops like open_pr_if_needed.

WHAT I'D TAKE

› Middleware-first orchestration · › Queued follow-up messages · › Persistent workspaces · › Deterministic safety nets

WHAT I'D CHANGE

› Keep local, not cloud-sandboxed · › Keep one writing agent · › Tighten edit verification

Claude Code

BEST FOR CONTEXT DISCIPLINE

DOCS STUDIED

How it works · Memory · Custom subagents

HOW IT EDITS

The public docs focus on checkpoints and permissions more than on the exact edit primitive. The important operational pattern is checkpointing before edits and allowing safe undo.

CONTEXT MANAGEMENT

Uses CLAUDE.md for persistent project instructions. Uses auto memory for learned preferences and corrections. Compacts context automatically when sessions get long. Uses isolated subagent contexts that return summaries.

ERROR HANDLING

Uses permission modes to limit autonomous actions. Background subagents fail closed on unapproved permissions. Session edits can be undone via checkpoints.

WHAT I'D TAKE

› Persistent repo instructions · › Checkpoints before edits · › Isolated subagent contexts · › Explicit permission modes

WHAT I'D CHANGE

› Keep the system smaller and more explicit · › Document the edit strategy directly

2 FILE EDITING STRATEGY — ANCHOR-BASED REPLACEMENT

I am adopting anchor-based replacement as the primary file editing strategy. The mechanism works as follows: read the file, choose a unique surrounding text block as the anchor, replace only that anchored block, then verify match count, diff size, and post-edit correctness. This approach is simpler than AST editing, safer than line numbers because it does not depend on drifting offsets, easier for the model to produce reliably than perfect unified diffs, and works uniformly across TypeScript, JSON, SQL, Markdown, CSS, and configuration files.

• WHY THIS STRATEGY

✓ Simpler than AST editing
✓ Safer than line numbers — no drifting offsets
✓ Easier for models than perfect unified diffs
✓ Works across TS, JSON, SQL, Markdown, CSS, config

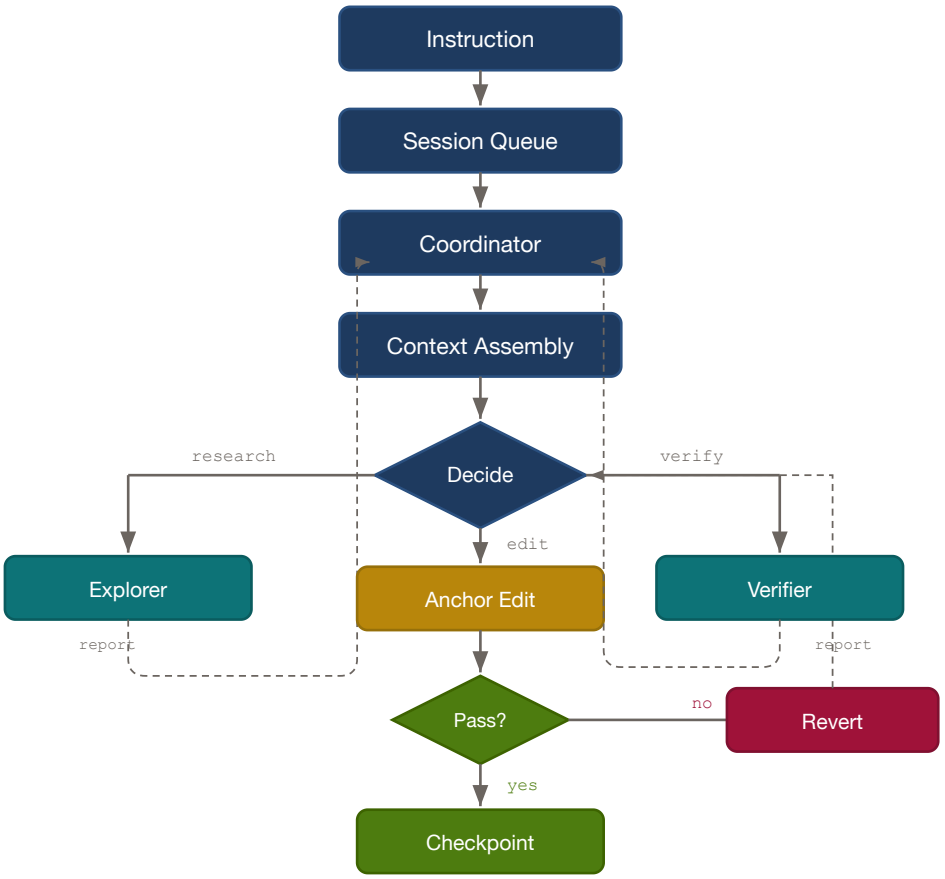
• WHY NOT THE OTHERS

✗ Unified diff — great for audit, too formatting-fragile
✗ Line-range — too brittle when files shift
✗ AST editing — too expensive to build correctly across all file types

The main failure modes are: the anchor is not unique (zero-match or multi-match), the file changed after read, the edit is syntactically valid but logically wrong, or the edit is too large and behaves like a file rewrite. Each of these is handled by specific guardrails: reject zero-match and multi-match edits, re-read if the file hash changed, run targeted verification after each edit, and reject oversized diffs and retry with a narrower anchor.

3 SYSTEM ARCHITECTURE

The system architecture follows a coordinator-centric model. The user instruction enters a persistent session queue. The coordinator loads state, assembles context, and decides the next action. Read-only subagents can gather context or verification results in parallel. Only the coordinator writes files. Every edit goes through verification and recovery before the run is considered complete.



4 EDITING STEP-BY-STEP

Every file edit follows an eight-step process. The goal is to make each edit as narrow and verifiable as possible, with explicit checkpoints for recovery. If any step fails, the system knows exactly where to revert to and what to retry.

1	**Read** the target file and record a content hash
2	**Identify** the smallest unique anchor block around the change
3	**Confirm** the anchor appears exactly once
4	**Generate** a replacement only for that block
5	**Preview** the diff — reject if too large or touches unrelated code
6	**Apply** the edit
7	**Verify** immediately — syntax/type-check, targeted tests, diff guard
8	**Checkpoint** and continue to the next task

Recovery Paths

When things go wrong, the recovery logic depends on the specific failure type. Zero matches trigger a re-read with a better anchor. Multiple matches trigger anchor expansion. A changed file means discarding the old plan and rebuilding from the latest version. Verification failure triggers a revert and single retry. Repeated failure escalates instead of thrashing.

Zero matches	→ Re-read file + try better anchor
Multiple matches	→ Expand anchor and retry
File changed	→ Discard plan + rebuild from latest
Verification failed	→ Revert to checkpoint + retry once
Repeated failure	→ Escalate to user or Opus

No full rewrites
A bad edit never justifies rewriting the whole file.

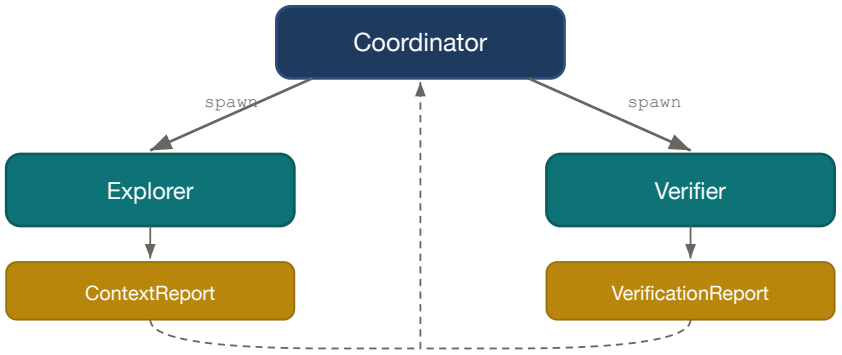
No stale edits
Never edit a file that wasn't read in the current cycle.

No oversized diffs
Reject changes that alter too much for a surgical edit.

5 MULTI-AGENT DESIGN

The orchestration model uses one coordinator, one explorer subagent, and one verifier subagent. The coordinator owns the persistent loop, owns planning, owns all writes, and merges outputs. The explorer does read-only repo or spec research and returns a structured ContextReport. The verifier does read-only validation and returns a structured VerificationReport.

Agents communicate only through typed artifacts, not through shared free-form memory. When merging outputs, verifier evidence beats speculation, direct file evidence beats inference, and latest file reads beat stale memory. This design satisfies the multi-agent requirement, avoids merge conflicts from multiple writers, and still gives real speedup through parallel research and verification.



COMMUNICATION RULES

- ✓ Only through typed artifacts
- ✓ Not through shared free-form memory
- ✓ Verifier evidence beats speculation
- ✓ Direct file evidence beats inference
- ✓ Latest file reads beat stale memory

WHY THIS DESIGN

- ✓ Satisfies multi-agent requirement
- ✓ Avoids merge conflicts from multiple writers
- ✓ Real speedup through parallel research
- ✓ Single writer prevents coordination bugs

6 CONTEXT INJECTION SPEC

I will inject context as a typed ContextEnvelope. The envelope carries four categories of context: stable repo context (AGENTS.md, package manager and scripts, architecture notes, coding conventions), task context (user instruction, PRD or spec excerpts, target paths, success criteria), runtime context (current diff, tool outputs, failures, command results), and session memory (last summary, prior failed attempts, known blockers).

Context is injected at five points: before planning, before each edit turn, before each repair turn, before each subagent spawn, and before final verification. The rules are: stable context is cached, runtime context is clipped to only what matters, file excerpts are preferred over full files, and old turns are summarized instead of replayed verbatim.

STABLE REPO

- AGENTS.md
- Package scripts
- Architecture notes
- Coding conventions

TASK

- User instruction
- PRD / spec excerpts
- Target paths
- Success criteria

RUNTIME

- Current diff
- Tool outputs
- Failures
- Command results

SESSION MEMORY

- Last summary
- Prior failed attempts
- Known blockers

7 CONTEXT INJECTION EXAMPLE

Below is a concrete example of a ContextEnvelope for an edit session. The task is to fix a failing sprint approval flow. The stable context includes AGENTS.md excerpts and build scripts. The runtime context includes file excerpts from the two target files and the specific error output. The session memory records that a previous attempt touched the wrong approval branch.

```
{
  "sessionId": "sess_123",
  "task": {
    "instruction": "Fix the failing sprint approval flow",
    "goal": "targeted fix with tests passing",
    "targetPaths": ["api/src/routes/weeks.ts", "web/src/pages/UnifiedDocumentPage.tsx"]
  },
  "stableContext": {
    "repoRules": "AGENTS.md excerpt",
    "scripts": ["pnpm --filter @ship/api test", "pnpm --filter @ship/web test"],
    "conventions": ["TypeScript", "pnpm workspace", "targeted tests only"]
  },
  "runtimeContext": {
    "fileReads": [
      { "path": "api/src/routes/weeks.ts", "excerpt": "..." },
      { "path": "web/src/pages/UnifiedDocumentPage.tsx", "excerpt": "..." }
    ],
```

```
    "errors": [{ "tool": "test", "message": "Expected 200, got 500" }]
  },
  "sessionMemory": {
    "summary": "Previous attempt touched the wrong approval branch",
    "retryCountByFile": { "api/src/routes/weeks.ts": 1 }
  }
}
```

8 TOOL INVENTORY

The agent needs ten tools to operate. These cover file reading, search, editing, command execution, git operations, subagent coordination, checkpoint management, and structured tracing. The primary edit tool is edit_block which implements the anchor-based replacement strategy.

TOOL	PURPOSE
`read_file`	File reads with content hashing for stale-read detection
`search_files`	Fast symbol and content discovery across the codebase
`list_files`	Path discovery and directory traversal
`edit_block`	Anchor-based replacement — the primary edit tool
`write_file`	New file creation only (not for editing existing files)
`run_command`	Tests, builds, lint, type-check, and git commands
`git_diff`	Audit and preview of changes before and after edits
`spawn_subagent`	Launch explorer or verifier workers in parallel
`checkpoint_revert`	Recovery to last known-good state after failed edits
`trace_event`	Structured tracing to LangSmith for observability

9 FRAMEWORK CHOICE — LANGGRAPH.JS + ANTHROPIC SDK + LANGSMITH

I am using LangGraph.js with the Anthropic TypeScript SDK and LangSmith tracing. Ship already uses TypeScript and already has LangGraph packages in the repo. LangGraph gives explicit orchestration, state, and branching. It is easier to trace and explain than a custom while-loop. It is smaller and faster to stand up than a full Open SWE style cloud runtime.

WHY LANGGRAPH

✓ Ship already uses TypeScript
✓ LangGraph packages already in the repo
✓ Explicit orchestration, state, and branching
✓ Easier to trace than a custom while-loop
✓ Smaller than full Open SWE cloud runtime

IMPLEMENTATION STACK

› **Runtime:** Node.js + TypeScript
› **Orchestration:** LangGraph.js
› **Model API:** Anthropic TypeScript SDK
› **Tracing:** LangSmith
› **Interface:** CLI REPL first, HTTP optional

10 PERSISTENT LOOP

The persistent loop will run as a long-lived local Node process inside the repo. You start it once with a command like `pnpm agent:dev`. The process stays alive after each completed task and accepts new instructions through the same session without restarting.

It stays alive by blocking on the next instruction event instead of exiting, keeping active session state in memory for speed, and persisting lightweight session snapshots after each turn so the process can recover if restarted.

11 TOKEN BUDGET

The default model is Claude Sonnet 4.5 ($3/MTok input, $15/MTok output). The escalation model is Claude Opus 4.6 ($5/MTok input, $25/MTok output), used only for hard architectural reasoning or repeated repair failure. The budget policy is to keep normal turns under roughly 40k input tokens, compact or summarize before exceeding that budget, and keep outputs tight by avoiding replaying long logs.

PLAN / RESEARCH	EDIT TURN	REPAIR TURN
~12k	~20k	~35k
input · ~1.5k output	input · ~4k output	input · ~6k output

The main cost cliffs to avoid:

- ⚠ Including full large files in prompt instead of targeted excerpts
- ⚠ Including full test logs instead of just the failing slices
- ⚠ Spawning too many subagents with duplicated context
- ⚠ Repeated retries with the same oversized prompt
- ⚠ Escalating to Opus too early before Sonnet has been given a fair chance

12 BAD EDIT RECOVERY

When the agent makes a bad edit, detection comes from five signals: zero-match or multi-match anchor failure, file hash mismatch after read, syntax or type-check failure, targeted test failure, and diff guard failure. The important rule is that a bad edit does not justify rewriting the whole file.

The recovery flow is: stop at the failed edit, revert to the last checkpoint, re-read the file, rebuild the edit plan, retry once with a narrower anchor, and escalate if the second attempt still fails.

13 TRACING SPEC

The tracing system uses LangSmith as the primary trace system with a local JSONL log as the fallback artifact. Every run logs: session ID, instruction, branch and git snapshot, selected context inputs, subagent launches and results, model turn summaries, tool calls and outputs, file hashes before and after edits, preview diffs and applied diffs, verification commands and outputs, retry reasons, final outcome, elapsed time, and token estimate with model used.

A complete edit trace shows: instruction received → session restored → context selected → subagent research launched (optional) → edit target chosen → anchored edit applied → verification executed → pass/fail recorded → checkpoint revert + retry (if needed) → final result returned.

> **Final Architecture Decision:** I am building a TypeScript LangGraph agent with a persistent local loop, one writing coordinator, read-only specialist subagents, anchor-based surgical editing, and LangSmith traces. This is the best fit for the Ship repo and the Shipyard grading criteria.

Sources

LOCAL SOURCES

› Shipyard PRD
› Ship AGENTS.md
› Ship CONTEXT.md
› Ship WORKFLOW_MEMORY.md
› Ship IMPLEMENTATION_STRATEGY.md
› Ship root, API, and web package.json

EXTERNAL SOURCES

› OpenCode — README, edit.go, patch.go, agent.go
› Open SWE — server.py, middleware
› Deep Agents — graph, filesystem, summarization
› Claude Code — architecture, memory, subagents
› Anthropic pricing